

Contents

1. Introduction	1
2. Theoretical foundations and architecture	2
2.1. Formal model of the event log	2
2.2. Formal model of transformation rules: centralized approach	3
2.3. Formal Model of Transformation Rules: Compositional Approach	4
2.3.1. Limitations of the monadic model	5
2.4. Cache irrelevance requirement for computational determinism	6
3. Prototype implementation and evolution of the computation model	7
3.1. Haskell as a rule language and data model agnosticism	8
3.2. Resource architecture	8
3.3. From memoization to incremental automata	9
3.4. Gear Composition	11
3.5. StateGraph and retroactive changes	12
3.6. Prototype testing	13
3.6.1. Domain model description	13
3.6.2. Verification methodology and results	14
3.7. Analysis of prototyping results and defining further work	16
4. Development of the Faden language	17
4.1. Architectural Principles and Semantics	18
4.2. Dependent types	18
4.3. Typing	18
4.4. Records	19
4.5. Totality	20
4.6. Formal verification mechanism and rewrite rules	20
5. Conclusion	21
Bibliography	21
6. Faden Code Example	22

1. Introduction

Modern computer systems employ database management systems (DBMSs) to store and organize large volumes of data. Most modern production solutions (PostgreSQL, MySQL, Oracle) operate as imperative interfaces to the data. Under this approach, external systems (clients) send pointwise update (UPDATE, INSERT) and read commands to the DBMS.

However, in such systems the DBMS possesses limited knowledge of the semantic relationships within the data. This leads to several fundamental problems:

1. **Integrity problem.** Enforcement of complex invariants is delegated to external applications. The DBMS, acting as a passive data storage, cannot guarantee the preservation of domain-specific invariants.
2. **Verification difficulty.** Simple data updates may require executing long sequences of heavy instructions, the correctness of which is difficult to prove formally.
3. **Data loss risk.** An incorrect sequence of imperative commands can lead to irreversible data corruption.
4. **Centralization.** The imperative architecture complicates state synchronization between replicas, forcing many solutions to rely on a centralized node for all write operations.

This work proposes a **functional-reactive architecture** as an alternative to the imperative approach. The key features of the proposed model are:

- The architecture does not provide direct operations to delete or overwrite existing data. Interaction with the DBMS is limited to appending records to an event log (append-only).
- The DBMS is **event-sourced**: an ordered list of events serves as the single source of truth.
- The DBMS automatically organizes data based on developer-defined rules, transforming the event list through a multi-stage process into a final representation, available for client queries.

2. Theoretical foundations and architecture

2.1. Formal model of the event log

A trivial model representing history as a linear list of events $L = [e_1, e_2, \dots, e_n]$ is inadequate in a distributed environment due to the absence of globally synchronized time and the need to merge data from independent nodes.

To construct a robust model, we introduce the notion of an **event identifier** $\text{id} \in \mathcal{I}$, serving as the primary key of an event.

Set $\mathcal{I}$ must be **linearly ordered**. As an implementation of $\mathcal{I}$, one may consider lexicographically ordered tuples:

$$\text{id} = \langle t, n, c \rangle$$

where:

- t — physical time (Unix timestamp) of the event's registration on the source node;
- n — unique identifier of the source node;
- c — monotonically increasing counter of the source node, resolving collisions within a single time quantum.

The ordering may be defined as follows:

$$\begin{aligned} \text{id}_A < \text{id}_B &\Leftrightarrow (t_A < t_B) \\ &\vee (t_A = t_B \wedge n_A < n_B) \\ &\vee (t_A = t_B \wedge n_A = n_B \wedge c_A < c_B) \end{aligned}$$

Then the **event log** $\mathcal{L}$ is defined as a finite partial map from the set of identifiers $\mathcal{I}$ to the set of event payloads E :

$$\mathcal{L} : \mathcal{I} \rightharpoonup E$$

This definition possesses two fundamental advantages:

1. **Simplicity of merging.** Merging histories from two DBMS instances $\mathcal{L}_A$ and $\mathcal{L}_B$ reduces to the set-theoretic union of their graphs: $\mathcal{L}_{\text{merged}} = \mathcal{L}_A \cup \mathcal{L}_B$
2. **Consistency.** Although events may arrive in arbitrary order, the merge is deterministic, and its records possess a strict linear order.

A binary search tree using identifiers $\mathcal{I}$ as keys and storing payload E as values can serve as an implementation of this event log model.

2.2. Formal model of transformation rules: centralized approach

The proposed system is based on abandoning the storage of current state as a primary entity. Instead, the database state $\mathcal{S}$ at any given moment is determined by a function of the event history $\mathcal{L}$.

A naïve approach involves directly defining the function $f : \mathcal{L} \rightarrow \mathcal{S}$. However, as the number of events in the system grows, full state reconstruction becomes computationally infeasible.

To address the performance issue, we transition to a model of **incremental computation**. We define a **rule** in a manner that permits processing not of the entire source object in each invocation, but only of its “increment” relative to the previous query.

For instance, when the source object is an event log, the increment can be considered as merely the list of newly arrived events. Such incremental processing of incoming data enables reuse of previous computation results.

We define the set of internal rule states as $\mathcal{C}$ (“cache”). Then, an incremental rule can be represented as a function:

$$\text{rule}_{\text{top}} : \mathcal{C} \times \mathcal{L} \rightarrow \mathcal{C} \times \mathcal{S}$$

Formal correctness criteria for the internal state $\mathcal{C}$ will be discussed in more detail in section Section 2.4.

This representation is monolithic and centralized: the entire DBMS state is determined by a single high-level rule. This approach is formally correct and finds application in several functional-reactive programming models (for example, such as Monadic Stream Functions [1]). However, in the context of a DBMS, it leads to excessive coupling of logic and a “top-down” approach that makes it difficult to isolate sequential processing stages.

2.3. Formal Model of Transformation Rules: Compositional Approach

This work proposes a **compositional approach** based on decomposing the overall transformation logic into a set of independent automata with encapsulated state. Such a model transforms the data materialization process into a transparent multi-stage pipeline.

A key requirement of the model is the ability to combine rules with different internal state structures into a single computation graph. To achieve this, the concrete type $\mathcal{C}$ must be hidden from the external rule signature, leaving only an interface for querying the materialized value.

We define the type $R(K, O)$, parameterized by the configuration type K and the output type O :

$$R(K, O) := \exists \mathcal{C}. (\mathcal{C} \times (\mathcal{C} \rightarrow \mathcal{M}_K(\mathcal{C} \times O)))$$

An arbitrary element of type $R(K, O)$ describes a single system rule and contains:

1. A hidden internal state type $\mathcal{C}$, inaccessible to external consumers.
2. An initial state $c_0 \in \mathcal{C}$.

3. A transition function (“step”), operating within the monadic context $\mathcal{M}_K$. It accepts the current state $c \in \mathcal{C}$ and returns an updated state combined with the computation result.

Note that the transition function does not accept the event log explicitly. Instead, access to the event log and the results of other rules is provided through the monadic interface $\mathcal{M}_K$.

The monad $\mathcal{M}_K$ acts as an operational environment parameterized by type K , associated with the following primitives:

- **Materialization of other rules’ results:** $\text{resolve} : R(K, O') \rightarrow \mathcal{M}_K(O')$ — materializes the value of another rule. Allows a rule to utilize the computation results of other rules.
- **Reading the event log:** $\text{events} : \mathcal{M}_{K(\mathcal{L})}$ — returns a snapshot of the event log $\mathcal{L}$ at the moment the operational environment was launched due to external query.
- **Reading current configuration parameters:** $\text{ask} : \mathcal{M}_K(K)$ — returns the current configuration parameters of type K . This mechanism allows dynamic reconfiguration of an existing rule’s behavior without resetting its internal state.
- **Local configuration update:** $\text{local} : (K \rightarrow K') \rightarrow \mathcal{M}_{K'}(\mathcal{O}) \rightarrow \mathcal{M}_K(\mathcal{O})$ — invokes a monadic sub-computation with an updated configuration. This allows higher-level rules to exert limited control over the behavior of lower-level rules.

Formally, the monad $\mathcal{M}_K$ can be represented as a stack of monad transformers combining a configuration reading context (Reader monad), a rule state registry management context (State monad), and a context for accessing the current event log snapshot (Reader monad).

2.3.1. Limitations of the monadic model

Implementing the operational environment using the monad $\mathcal{M}$ provides maximum expressiveness (with the ability to dynamically choose further system behavior based on previous steps), but imposes certain constraints on memory management efficiency and garbage collection.

The dynamic nature of monadic composition introduces the following challenges:

1. **Opacity of the computation graph.** The inability to construct a static dependency graph complicates preliminary query optimization and the parallel execution of independent rules.
2. **Complexity of cache lifecycle management.** In a dynamic model, it is difficult to determine when the internal state of a specific rule becomes redundant and can be safely evicted from memory.

A promising solution under consideration is a complete or partial transition to **applicative functors** or **selective functors** (Selective Applicative Functors [2]) for the static description of dependency structure. This approach allows describing the transformation graph declaratively, prior to data processing, which opens possibilities for:

- Static analysis of memory consumption and deterministic garbage collection.
- Automatic parallelization of computations.

The current study uses the monadic model as the most flexible and simplest for the prototyping stage; however, the question of transitioning to a strict applicative interface remains an open problem for further research.

2.4. Cache irrelevance requirement for computational determinism

The introduction of internal rule state ($\mathcal{C}$) and the use of the monadic context $\mathcal{M}_K$ enables the description of non-trivial incremental computations, but creates a risk of violating the functional purity of the system. If the result of a rule's computation depends on the cache contents, the DBMS loses its determinism property, and the materialized state ceases to be unambiguous.

For example, if the system is sensitive to the order in which events arrive in the log, then synchronizing event logs from several independent DBMS instances may result in different final states.

To address this problem, we introduce the **cache irrelevance** requirement.

Let Γ be the set of state registries for all rules in the system. We denote the process of materializing rule $R(K, S)$ in configuration K based on event log $\mathcal{L}$ with initial registry Γ as the function:

$$\text{eval} : R(K, S) \times \mathcal{L} \times K \times \Gamma \rightarrow S \times \Gamma$$

We say that rule $r \in R(K, S)$ satisfies the cache irrelevance property if for any log $l \in \mathcal{L}$, configuration $k \in K$, and any two arbitrary registry states $\gamma_1, \gamma_2 \in \Gamma$, the following identity holds:

$$\text{fst}(\text{eval}(r, l, k, \gamma_1)) = \text{fst}(\text{eval}(r, l, k, \gamma_2))$$

where fst is the projection extracting the first element of the direct product $S \times \Gamma$.

Compliance with the cache irrelevance requirement is critically important for ensuring the integrity of the materialized state. However, verifying it in general-purpose languages (such as C++, Haskell, Rust) is associated with fundamental difficulties. These languages lack the necessary tooling for formal verification that would allow expressing the cache irrelevance requirement for a rule at the language level, describing its proof, and automatically checking its correctness. Correctness verification is delegated to external tools, whose full integration into the build process is problematic.

This work proposes using the formal verification apparatus [3], based on the Curry–Howard correspondence. Under this approach, the cache irrelevance property can be represented as a data type, and the proof of a rule’s compliance with the requirement is expressed as an inhabitant of this type.

Verification of the presented proof’s correctness can be performed by the compiler, significantly reducing the chance of human error. Thus, in a language supporting formal verification, the developer can not only describe an incremental transformation function $r \in R(K, O)$, but also statically guarantee its equivalence to some “reference” pure function $f : \mathcal{L} \times K \rightarrow O$.

The Curry–Howard correspondence also allows guaranteeing required invariants by imposing constraints on the configuration K , the internal state of a specific rule $\mathcal{C}$, and the materialized type O .

The ability to shift the responsibility for invariant checking from the human to the compiler is a strong argument in favor of using a specialized language for describing internal DBMS rules.

3. Prototype implementation and evolution of the computation model

To validate the proposed DBMS architecture, a prototype was developed in Haskell. This choice is motivated by the advanced type system, support for

algebraic data types, and functional purity, which ideally align with the concept of functional-reactive data processing.

This chapter is devoted to the development process of the prototype and a critical analysis of the made architectural decisions.

3.1. Haskell as a rule language and data model agnosticism

The key architectural decision that shaped the prototype was the early abandonment of an intermediate rule language. Instead, ordinary functions of the implementation language (Haskell) were used for defining rules and performing transformations.

This decision enabled the prototype to:

- Leverage the full power of a general-purpose language for defining rules.
- Ensure data model agnosticism. Most industrial DBMSs impose a rigid data organization format (e.g., relational, document-oriented, graph-based), which often creates a mismatch between in-memory data structures of the client application and their storage format in the database.

In the developed prototype, the DBMS is agnostic with respect to the specific data format: the system operates not on tuples or documents, but on arbitrary serializable objects (e.g., trees). Thus, the database schema is determined exclusively by the type system of the rule language used.

A drawback of this decision was the need to develop mechanisms within the prototype for serializing arbitrary objects. The most difficulty was posed by closures, which cannot be serialized using Haskell's built-in mechanisms.

3.2. Resource architecture

A direct consequence of abandoning a fixed data format was the need to implement a comprehensive memory management subsystem. Since the DBMS operates on arbitrary inductive data types (trees, lists, graphs), the problem of efficient deserialization of large objects arises.

Consider the task of retrieving a single record from a large table by primary key. Classical relational DBMSs handle this scenario efficiently by partially loading the table into main memory, leveraging paged organization.

On the other hand, the prototype treats the entire table as a single Haskell object (e.g., one large balanced tree) supporting individual record lookup operations. Full loading and deserialization of such an object requires loading the entire

table, whose size potentially exceeds the available memory — unjustified when performing a point query for a single record.

To address this problem, a mechanism for partial loading and deserialization of deeply nested structures is necessary.

For organizing data residing at different levels of the memory hierarchy, the prototype introduced the `Res` abstraction. `Res` acts as a smart pointer, abstracting the physical location of data and providing on-demand loading and deserialization of heavy data structures.

$$\text{Res}(A) := \text{ResInline}(A) \mid \text{ResBuiltin}(\text{Word64}, A) \mid \text{ResAlloc}(\text{ResA}(A))$$

where:

- `ResInline` — direct inline storage of the resource.

Applicable for simple resources that do not require separate allocations.

- `ResBuiltin` — a built-in resource with a static identifier (a 64-bit number).
- `ResAlloc` — a dynamic resource that potentially requires deserialization upon first access.

A widely known technique in functional programming is the use of lazy evaluation and memoization. In Haskell, the lazy evaluation model enables the construction of “infinite” data structures generated on demand. Evaluated subparts of the “infinite” structure are retained and not recomputed. A classic example of such an “infinite” data structure is the Haskell implementation of the Fibonacci sequence list, shown in listing Listing 1.

```
fibs :: [Integer]
fibs = 0 : 1 : zipWith (+) fibs (tail fibs)
```

Listing 1: Lazy infinite list implementation of the Fibonacci sequence in Haskell

To achieve a similar effect within the DBMS, the smart pointer `Delay` was implemented as an extension of `Res`, supporting deferred computations and caching of their results.

While `Delay` achieved the desired effect, the time overhead and unpredictable memory usage associated with lazy evaluation make `Delay` impractical and hazardous in real-world scenarios.

3.3. From memoization to incremental automata

The initial data transformation concept relied on a mathematically elegant but practically infeasible model of full history projection:

$$f : \mathcal{L} \rightarrow S$$

Under this model, it was assumed that the lazy nature of Delay and memoization would allow the system to automatically cache intermediate computation results. For example, if the event log $\mathcal{L}$ grows by one event e , the lazy computation tree should update only those nodes that directly depend on e , reusing old computations for other parts of the tree thanks to memoization.

A more detailed analysis revealed that:

- Full memoization is extremely memory-intensive;
- Not all transformations are trivially representable through recursion;
- The recursive approach to describing transformations complicates the processing of primitive updates.

To overcome these limitations, a transition was made to a more flexible model of explicit state management, implemented through the Gear abstraction. The Gear concept represents rules as state machines with explicit internal state, rather than pure functions of the event history. In this model, a rule possesses an explicit internal state type $\mathcal{C}$ and a transition function operating within a monadic context.

However, implementing this model revealed a fundamental problem of dynamic configuration. In real systems, the need arises to change system behavior at runtime, and any change to the function code in a naïve architecture requires a complete reset of the internal state and recomputation of the entire history.

To address this challenge, a structure was developed that supports a configuration substitution mechanism without losing the internal state. The final Gear implementation in the prototype is shown in listing Listing 2.

```
data GearTemplate ctx (out :: Type) cache cfg = UnsafeGearTemplate !
  cache !((ctx, W1 Maybe cfg) :-> M AppIOc cfg) !((cfg, cache) :-> M
  AppIOc (out, cache))
  deriving (Generic)

data GearFn ctx out cache = ∀ cfg. GearFn !(ValT cfg) !cfg !
  (GearTemplate ctx out cache cfg)

data Gear ctx out = ∀ cache. UnsafeGear !(ValT cache) !(GearFn ctx
  out cache) !Int
```

Listing 2: Implementation of the Gear abstraction in the prototype

GearTemplate consists of three components:

1. **Initial internal state cache.**
2. **Initialization/migration function:** The function $(ctx, \text{wl Maybe } cfg) \rightarrow M \text{ AppI OC } cfg$ accepts the current context and, potentially, the previous configuration, computing a new configuration cfg .
3. **Step function:** The function $(cfg, cache) \rightarrow M \text{ AppI OC } (out, cache)$ performs the update of the internal store and produces an object of type out .

This architecture achieved high computational efficiency: upon receiving new events, the system does not recompute the entire state but only invokes the step function for affected rules. The use of existential types $(\text{forall } cfg \text{ cache})$ enabled the creation of a rule tree in which each rule encapsulates its own internal configuration and state.

Note that this implementation of reactive cells permits defining rules using recursion and memoization, so this model is a more powerful version of the original concept $f : \mathcal{L} \rightarrow S$.

3.4. Gear Composition

Despite the flexibility of the `GearTemplate` model, manually describing all three components (initial state, migration function, and step function) for each rule involves writing a large amount of boilerplate code. This problem is especially acute when attempting to combine several simple rules into one complex one: the developer must manually describe product types for combining caches $(\mathcal{C}_1 \times \mathcal{C}_2)$ and configurations $(K_1 \times K_2)$.

To solve this problem, the prototype developed the applicative structure `Asm` (“Assembly”), which manages the lifecycle of hierarchical states.

From the perspective of category theory, `Asm` is an applicative functor with defined `pure` and `<*>` operations. This enables declarative rule composition:

$$\text{pure} : O \rightarrow \text{Asm}(\text{Ctx}, O)$$

$$<*> : \text{Asm}(\text{Ctx}, O_1 \rightarrow O_2) \times \text{Asm}(\text{Ctx}, O_1) \rightarrow \text{Asm}(\text{Ctx}, O_2)$$

Under this decomposition, the DBMS engine automatically constructs the state and configuration tree corresponding to the `Asm` composition tree. This guarantees that each sub-rule receives exactly the portion of the overall state and configuration that belongs to it, ensuring strict isolation and type safety.

One of the most expressive examples of using this abstraction is the buffered combinator. It allows any rule to “remember” its previous result, transforming it from an instantaneous value into a “previous-current” pair:

$$\text{buffered} : O_{\text{init}} \rightarrow \text{Asm}(\text{Ctx}, O) \rightarrow \text{Asm}(\text{Ctx}, O \times O)$$

The buffered implementation (listing Listing 3) employs the `asmCached` primitive, which allocates an additional memory cell in the internal state hierarchy for storing the value from the previous iteration.

```
buffered :: (InferValT True out) => out -> Asm ctx out -> Asm ctx
(out, out)
buffered cache0 outA = asmCached cache0 do
  out <- outA
  pure $ \old -> pure ((old, out), out)
```

Listing 3: Implementation of the buffering combinator based on Asm

This combinator proves extremely useful for implementing change tracking between old and new versions, upon which many incremental rules are built.

3.5. StateGraph and retroactive changes

One of the fundamental challenges in event-sourced systems is the correct handling of retroactive changes and maintaining data consistency regardless of the order in which events arrive.

Suppose rule r_a , when computing the state of object x at time t_2 , reads the state of object y computed by rule r_b at time t_1 . If a new event e arrives in the system, changing the state of y at time t_1 , the system must initiate recomputation of the state of x .

To address this challenge, the StateGraph mechanism was developed in the prototype. Its primary task is tracking the dependency graph between the states of various objects and automatically performing recomputation for all objects indirectly affected by the current update.

The StateGraph implementation in the prototype stores a chronology for each object, including all its state changes and the moments when an object’s state was read to form the state of another object. A compressed prefix tree is used for storing the chronology, enabling efficient extraction of only the range of records following the update point.

The update propagation algorithm is implemented through a recomputation queue. When a state changes at a graph node, all associated “listener events” are placed in a shared queue and recomputed.

3.6. Prototype testing

For the final verification of the correctness of the implemented mechanisms (Res, Gear, Asm, StateGraph), a test scenario was developed modeling a role-based access control (RBAC) system based on events.

3.6.1. Domain model description

An access control system was chosen as the test model. Listing Listing 4 presents the set of roles SiteAccessLevel and the set of events Event.

```
data SiteAccessLevel = SalNone | SalUser | SalModerator | SalAdmin
  deriving (Eq, Show, Generic)

data Event
  = CreateUser -- Registration of a new user
  | AdminSetAccessLevel !(W1 Maybe UserId) !UserId !(W
    SiteAccessLevel)
```

Listing 4: Fragment of the test scenario data model

Using StateGraph, the status rule was implemented, tracking and updating the state of all users. The source code of the rule is shown in listing Listing 5.

When an access level change event AdminSetAccessLevel arrives in the system, a check is performed to verify that the event’s initiator (adminM) is absent (meaning the event was sent by a superuser), or exists and possesses administrator privileges (SalAdmin). Reading the current state of the initiator is performed using the SG.query command, and the result is recorded via SG.update.

```

status = SG.mkStateGraph (asmGear events) (SG.StateGraphDepsNil,
\case
  AdminSetAccessLevel (W1 adminM) target level -> do
    hasAccess <- case adminM of
      Nothing -> pure True -- Superuser
      Just admin -> SG.query admin <&> \case
        Just (W SalAdmin) -> True
        _ -> False
    when hasAccess $ SG.update target $ pure level
  _ -> pure ()
)

```

Listing 5: Implementation of the access verification rule via StateGraph

3.6.2. Verification methodology and results

To confirm the correctness of retroactive update mechanisms, a test was developed using the QuickCheck library. The primary testing objective is to verify that the materialized system state is invariant with respect to the order of event arrival while preserving EventId labels.

Listing Listing 6 presents the key testing functions:

- oneshot (processing the event log in its entirety);
- multishot (incremental processing of random subsets of the log).

```

oneshot :: [(EventId, W Event)] → (SG.StateGraph EventId (W
SiteAccessLevel) → AppIOC r) → r -- [(UserId, [(EventId,
SiteAccessLevel)])]
oneshot t renderer = unsafeRunAppIO do
  status' ← confNewGear status ()
  putEventList t
  renderer =<< runGear status'

prop_test1_oneshot_correct =
  withMaxSuccess 1
    $ test1Res
    == oneshot test1 SG.toLists

multishot :: [(EventId, W Event)] → (SG.StateGraph UserId (W
SiteAccessLevel) → AppIOC a) → a
multishot t renderer = unsafeRunAppIO do
  status' ← confNewGear status ()
  for_ @[] (inits t) \curr → do
    _ ← runGear status'
    putEventList curr
  renderer =<< runGear status'

```

{- | This test shuffles the list of events and provides events to Dentrado one by one. It is expected that any shuffle of the input events yields the same result, since all events are associated with some point in time. Dentrado, being reactive, processes these events incrementally, but might perform expensive history rewrites to keep the result consistent. -}

```

prop_test1_multishot_correct = withMaxSuccess 100 $ forAllShrink
  (shuffle test1)
  (shrinkList shrinkNothing)
  \test1' →
    oneshot test1' SG.toLists == multishot test1' SG.toLists

```

Listing 6: Formulation of the order invariance property in QuickCheck terms

Testing was conducted on a chain of dependent events including:

1. Registration of multiple users.
2. Granting of administrator privileges to a user by the superuser.
3. Rejected operations to update other users' statuses by a non-privileged user.
4. Successful privilege management operations by an administrator.

The test results log is attached in listing Listing 7.

```
Running 1 test suites...
Test suite model: RUNNING...
=== prop_test1_oneshot_correct from tests/Model.hs:49 ===
+++ OK, passed 1 test.

=== prop_test1_multishot_correct from tests/Model.hs:68 ===
+++ OK, passed 100 tests.

Test suite model: PASS
Test suite logged to:
1 of 1 test suites (1 of 1 test cases) passed.
```

Listing 7: Prototype test results log

This log demonstrates how, over 100 experiments with random permutations of the original test sequence, the system converged to the same final state.

3.7. Analysis of prototyping results and defining further work

The prototype implementation in Haskell confirmed the theoretical viability of the proposed architecture. The mechanisms of incremental computation (Gear), composition (Asm), retroactive changes, and consistency maintenance (StateGraph) were successfully implemented. However, during development, limitations associated with using a general-purpose language for describing DBMS logic were identified:

1. **Function serialization problem.** While the functional paradigm permits the use of higher-order functions, their practical application in the prototype proved difficult due to the impossibility of serializing native Haskell closures using built-in mechanisms. To partially circumvent this problem, static function registries had to be employed, which are difficult to maintain. It is desirable to use a language for the DBMS where arbitrary functions are serializable and can be directly stored on disk or transmitted over the network.
2. **Lack of formal verification tooling.** While Haskell's type system allows expressing complex invariants, it is not designed for full-scale theorem proving. To implement the "cache irrelevance" requirement (described in section Section 2.4), it is necessary to have the ability to prove at the type level the equivalence of an incremental algorithm and its pure specification.

The absence of formal verification support in the language leaves a risk of desynchronization between the materialized state and the source event log.

3. **Lack of totality guarantees.** Haskell is a Turing-complete language, which permits the erroneous definition of rules with non-terminating computations. Thus, an incorrectly formulated rule in the context of a DBMS can cause the failure of the entire system. It is desirable that the language used can guarantee computation totality.
4. **Cyclic data structure problem.** Haskell permits the construction of cyclic data structures. However, the presence of cycles in the data graph significantly complicates disk space organization and the garbage collection mechanism. For efficient operation, it is advisable to restrict the DBMS to rules producing exclusively directed acyclic graphs.
5. **Complexity.** Haskell is a general-purpose language providing tools for solving a wide range of problems. For the specific task of describing data transformations, such universality is excessive. Based on these findings, the task of developing a specialized functional programming language for describing DBMS rules is formulated.

Based on these limitations, the following requirements were formulated for the language to be used for defining and verifying rules:

- **Minimalism and ease of learning.**
- **Full serializability of all entities**, including closures.
- **Dependent typing and formal verification capabilities** for supporting correctness proofs of transformations.
- **Totality guarantee** for eliminating infinite loops at the type-checking stage.
- **Data acyclicity:** prohibition of cyclic references at the language semantics level.

Based on the formulated requirements, the Faden language was designed. Its architectural principles and features are discussed in the following section.

4. Development of the Faden language

To overcome the identified limitations, the specialized language Faden was developed. It is a functional declarative language with gradual, extrinsic dependent typing.

The language is designed so that rule code can be stored, transmitted over the network, and executed in an isolated DBMS environment, while providing mathematical guarantees of the correctness of performed operations. An

example of Faden code with a general demonstration of syntactic constructs is presented in appendix Section 6.

4.1. Architectural Principles and Semantics

Fadeno is inspired by the Dhall and Cedille languages [4].

Its foundations include the following principles:

1. **Everything is an expression.** The language has no distinction between statements and expressions. Any construct — from a numeric literal to a lambda function declaration — is an expression that returns a value. This simplifies parsing, increases modularity, and facilitates composition.
2. **Extrinsic typing and erasure, gradual typing.** Types in Fadeno and certain constructs exist exclusively for type-checking and are erased at subsequent compilation stages, without affecting them. This approach establishes the role of static typing solely as a safety guarantee. The gradual typing model allows the language to function even if the program does not pass full type checking. This simplifies rapid prototyping, makes the compilation process predictable, and eases initial language adoption.
3. **Serializability.** All objects in Fadeno, including lambda functions, are serializable.
4. **Totality and formal verification support.** Fully typed programs in the language are total (guaranteed to terminate in finite time). The language supports functionality for theorem proving and functional verification.

4.2. Dependent types

The core of Fadeno is support for dependent types, where types can depend on values. In the context of a DBMS, this enables the description of complex data structures in which the typing of one part of the structure depends on the value of another.

An example of such a structure is a tagged union. A tagged union contains a numeric tag describing the variant and a value corresponding to that variant. A tagged union is an ideal demonstration of dependent types, as the type of the stored value depends on the value of the numeric tag.

4.3. Typing

Under nominative typing, two types are considered equivalent only if they originate from the same definition.

Under structural typing, two types are compatible if their internal structure matches, regardless of the type's origin.

Fadeno uses structural typing by default, permitting the use of nominative typing for encapsulating the internal structure of types.

Additionally, Fadeno supports the notion of subtypes, allowing objects of a narrower subtype to be used directly in a broader context without the need for manual conversion (e.g., using a natural number in a context expecting an integer).

Type-checking of expressions against a type is performed in bidirectional mode, where the compiler automatically checks an expression against a given annotation and automatically synthesizes a type for unannotated expressions.

These decisions significantly reduce the amount of boilerplate code and simplify the writing of logically correct applications.

4.4. Records

Working with data schemas is critically important for a DBMS. For storing document data, Fadeno employs records.

Records are a set of fields, where each field is labeled by a tag (a unique identifier starting with a dot and having the form `.custom_tag`) and may have a type different from others.

A “row” is an enumeration of the field list of the corresponding record at the type level. A field `.field` of a record `record` can be accessed using the syntax `record.field`.

The language implements **dependent records** and **row polymorphism** through an experimental dependent row concatenation construct (`\./`).

Dependent concatenation enables the construction of complex rows describing dependent records as combinations of simple rows.

An example of using dependent concatenation to construct a dependent record is shown in listing Listing 8. `Option(V)` is expressed as a dependent concatenation of a row with a single field `.has_value` of type `Bool` with a row whose structure is determined by the value stored in the `.has_value` field.

```
Option = \V. {( .has_value = Bool )} ./ (if ..has_value { .v = V }
{})
```

Listing 8: Implementation of the `Option` cell using a dependent record

Dependent row concatenation is a variation of dependent intersection [5] for defining dependent records. A distinctive feature of concatenation is predictable behavior, the ability to layer multiple fields with the same name on top of each other, and simplified type compatibility checking logic.

4.5. Totality

For use in the DBMS core, it is critical that user-defined rules are guaranteed to terminate (without falling into infinite loops). Faden is a **total** language.

Instead of uncontrolled recursion (the Y combinator), the language provides a built-in measure-based recursive primitive `loop`. Its signature is:

$$\begin{aligned} \text{loop} : & \text{Fun } \{I\} \{ \text{measure} : I \rightarrow \text{Int } + \} \{O\} \\ & (I) (\text{Fun } (\text{curr} : I) (\text{Fun } (\text{next} : I) \{ \text{measure next} < \text{measure curr} \} \rightarrow O) \rightarrow O) \rightarrow O \end{aligned}$$

This means that to define a loop, the developer must provide:

1. A metric function mapping the loop state to a non-negative integer (`Int+`).
2. A proof that the metric strictly decreases at each iteration.

Since a sequence of non-negative integers cannot decrease indefinitely, this statically guarantees program termination.

To avoid Girard's paradox arising from allowing `Type : Type`, Faden implements a hierarchy of type universes: instead of a single type `Type`, the language provides a built-in function `Type^` returning the type universe for each natural number:

$$\text{Type} = \text{Type}^0 : \text{Type}^1 : \text{Type}^2 : \dots$$

4.6. Formal verification mechanism and rewrite rules

Faden uses the Curry–Howard correspondence for theorem proving. The type `Eq a b` represents a proof of equality between expressions `a` and `b`.

A unique feature of the compiler is the support for user-defined rewrite rules (`rewrite`). If a proof `proof : Eq a b` exists in context, the construct `rewrite proof in expr` instructs the compiler to automatically substitute expression `b` for every occurrence of expression `a` within expression `expr`.

This is an extremely powerful tool for succinctly describing complex proofs.

To further simplify routine proofs, Fadenó supports a number of arithmetic simplifications at the normalization stage, employing the reduction of arbitrary algebraic expressions to polynomial normal form.

5. Conclusion

During this research project, a novel DBMS architecture was designed and prototyped.

Testing results on an RBAC model confirmed the invariance of state with respect to event ordering.

The developed language Fadenó enables shifting the verification of complex integrity invariants from the application level to the compiler, minimizing the data loss risks characteristic of imperative architectures.

Future work consists of implementing the functional-reactive DBMS architecture based on the Fadenó language and building the execution system.

Bibliography

- [1] I. Perez, M. Bärenz, and H. Nilsson, “Functional reactive programming, refactored,” *SIGPLAN Not.*, vol. 51, no. 12, pp. 33–44, Sept. 2016, doi: 10.1145/3241625.2976010.
- [2] A. Mokhov, G. Lukyanov, S. Marlow, and J. Dimino, “Selective applicative functors,” *Proc. ACM Program. Lang.*, vol. 3, no. ICFP, July 2019, doi: 10.1145/3341694.
- [3] A. Chlipala, *Certified Programming with Dependent Types: A Pragmatic Introduction to the Coq Proof Assistant*. The MIT Press, 2013. doi: 10.7551/mitpress/9153.001.0001.
- [4] A. Stump, “The calculus of dependent lambda eliminations,” *Journal of Functional Programming*, vol. 27, p. e14, 2017, doi: 10.1017/S0956796817000053.
- [5] A. Kopylov, “Dependent intersection: a new way of defining records in type theory,” in *18th Annual IEEE Symposium of Logic in Computer Science*, 2003. *Proceedings.*, 2003, pp. 86–95. doi: 10.1109/LICS.2003.1210048.

6. Faden Code Example

```
// Comments

// Unpacks Int, Type^, List, if... from built-in "faden" library
into current context
unpack faden. Int Type^ List if Eq refl Bool

// Constants
x = true

// Assigning lambda to constant
add =
  // Lambda of arguments "a" and "b"
  \a b.
  // Call to the builtin function `faden.int_add`, saving the
  result to constant `result`
  result = faden.int_add a b
  in result

// Type annotations for constants, checked
// If no annotation is provided, it's inferred automatically
/: Int
y = 4

// Type of functions is `Fun (A) (B) (C) -> Z`

// Function of two Int's that returns Int.
/: Fun (Int) (Int) -> Int
constf = \arg1 arg2.
  arg2 // returns the second argument passed

// Lists
/: List Int
list = [1 | 3 | 5 | 4 | -1]

// Records and row types
/: {( .field1 = Int | .field2 = Fun (Int) (Int) -> Int | .zer =
List Int )}
record =
  { .field1 = 4
  | .field2 = constf
  | .zer = [ 1 | 3 ]
  }
```

```

// Accesing fields of records
accessedField1 = record.field1

// Type aliases
/: Type^ 0
MyRecord = {( .a = List Int )}

/: MyRecord
my_record = { .a = [] }

// Type universes
/: Type^ 1
Type = Type^ 0
/: Type^ 2
Kind = Type^ 1

// Polymorphism
/: Fun {A} (A) -> A
id = \@A x.
  x

// Theorem proving capabilities
/: Fun {a} {b} {c} (Eq a b) (Eq b c) -> Eq a c
proof = \ab bc.
  rewrite ab
  in bc

// Dependent types
/: Fun (x : Bool) -> if x (Int) (List (List Int))
cond = \x.
  if x
    (\@x_is_true. rewrite x_is_true in 3)
    (\@x_is_false. rewrite x_is_false in [[3]])

// Importing other files
std = ./std

four = std._+_ 2 2

in {}

```